

Multi-Agent Technical Design for Governed GraphRAG

Vector DB + Neo4j Knowledge Graph + Text-to-Cypher + Skills + MCP Tools

Prepared by ChatGPT

April 30, 2026

Table of Contents

1. Executive Summary

This document defines a production-oriented multi-agent design for a governed GraphRAG platform using an existing Vector Database for embeddings and Neo4j as the knowledge graph. The platform allows users to ask natural-language questions and receive grounded answers based on graph relationships, retrieved semantic context, validated Cypher, and enterprise access controls.

The design is intentionally governed. Agents can reason, retrieve, generate, repair, and propose; only controlled services can execute queries or create governance issues. No LLM or agent receives direct unrestricted access to Neo4j.

Core design principle:

Vector search understands meaning.

Neo4j understands relationships.

Text-to-Cypher asks precise graph questions.

Validation and policy make it safe.

Result verification makes it trustworthy.

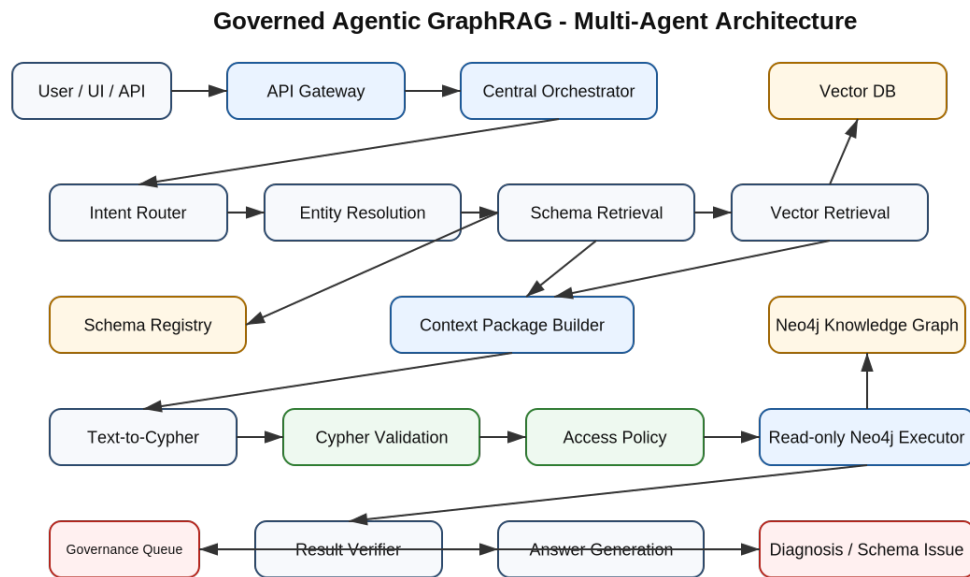
Agentic repair improves resilience without bypassing governance.

2. Target Outcomes

The system should support questions such as:

- Which internet-facing applications owned by Ravi have critical open vulnerabilities?
- Which applications have no active compensating control?
- Which business units have the most unresolved critical vulnerabilities?
- Which controls mitigate vulnerabilities for a given application?
- Why did this app not appear in the result?
- What schema or data issue prevented this query from working?

3. High-Level Architecture



Governed Agentic GraphRAG Architecture

3.1 Architecture Summary

User / App UI / API

- > GraphRAG API Gateway
- > Central Orchestrator
- > Intent Router Agent
- > Entity Resolution Agent
- > Schema Retrieval Agent + Vector Retrieval Agent
- > Context Package Builder
- > Text-to-Cypher Agent
- > Cypher Validation Agent
- > Access Policy Agent
- > Read-only Neo4j Executor
- > Result Verification Agent
- > Answer Generation Agent
- > Final Response

Failure paths are routed to:

Invalid Cypher -> Cypher Repair Agent -> Validation again

Empty or suspicious result -> Diagnosis Agent

Missing schema/data -> Schema Issue Agent -> Governance Queue

4. Multi-Agent Design

4.1 Agent Inventory

#	Agent	Purpose	MVP Required
1	Intent Router Agent	Classifies the question and selects retrieval/query strategy	Yes
2	Entity Resolution Agent	Resolves people, apps, CVEs, controls, business terms, values	Yes
3	Schema Retrieval Agent	Retrieves relevant Neo4j labels, relationships, properties	Yes
4	Vector Retrieval Agent	Retrieves semantic context from vector DB	Yes
5	Text-to-Cypher Agent	Generates schema-grounded read-only Cypher	Yes
6	Cypher Validation Agent	Validates safety, schema correctness, limits, and query risk	Yes
7	Access Policy Agent	Applies tenant, BU, entitlement, masking, and row-level filters	Yes
8	Result Verification Agent	Checks if query results answer the question	Yes
9	Answer Generation Agent	Produces final grounded answer	Yes
10	Cypher Repair Agent	Repairs invalid Cypher and schema mapping mistakes	Yes
11	Diagnosis Agent	Diagnoses empty/suspicious results	Phase 2
12	Schema Issue Agent	Creates schema/data quality issues for governance	Phase 2

#	Agent	Purpose	MVP Required
13	Feedback Learning Agent	Captures user corrections and proposes aliases/examples	Phase 3
14	Evaluation Agent	Runs golden-question regression tests	Phase 3

5. Agent Responsibilities

5.1 Intent Router Agent

Purpose: classify the user question.

Possible intent values:

vector_rag
graph_lookup
graph_traversal
graph_aggregation
hybrid_graphrag
schema_question
data_quality_question
clarification_required

Example output:

```
{
  "intent": "hybrid_graphrag",
  "requiresVectorSearch": true,
  "requiresCypher": true,
  "requiresGraphExpansion": false,
  "confidence": 0.93
}
```

5.2 Entity Resolution Agent

Purpose: map user language to real entities, graph labels, properties, and values.

Examples:

"apps" -> Application
"CVE" -> Vulnerability
"Ravi" -> Ravi Kumar / employeeId

"internet-facing" -> Application.exposure IN [External, Public, InternetFacing]
"critical" -> Vulnerability.severity = Critical

Example output:

```
{
  "entities": {
    "owner": {
      "input": "Ravi",
      "resolvedName": "Ravi Kumar",
      "resolvedId": "EMP-12345",
      "confidence": 0.88
    },
    "assetType": {
      "input": "applications",
      "resolvedLabel": "Application",
      "confidence": 0.97
    },
    "severity": {
      "input": "critical",
      "property": "Vulnerability.severity",
      "value": "Critical"
    }
  }
}
```

5.3 Schema Retrieval Agent

Purpose: retrieve the minimum relevant graph schema needed for the query.

Example output:

```
{
  "nodes": ["Person", "Application", "Vulnerability", "Control"],
  "relationships": [
    "(:Person)-[:OWNS]->(:Application)",
    "(:Application)-[:HAS_VULNERABILITY]->(:Vulnerability)",
    "(:Vulnerability)-[:MITIGATED_BY]->(:Control)"
  ],
  "properties": {
    "Person": ["name", "email", "employeeId"],
    "Application": ["name", "appId", "exposure", "businessUnit", "tenantId"],
    "Vulnerability": ["cveId", "severity", "status", "cvss"],
    "Control": ["name", "status", "controlId"]
  }
}
```

```
}  
}
```

5.4 Vector Retrieval Agent

Purpose: retrieve semantic context from vector DB.

Searches these indexes:

```
schema_index  
alias_index  
cypher_example_index  
document_chunk_index  
entity_index  
policy_index
```

Example output:

```
{  
  "semanticMappings": {  
    "internet-facing": "Application.exposure IN ['External', 'Public', 'InternetFacing']",  
    "CVE": "Vulnerability",  
    "open vulnerability": "Vulnerability.status IN ['Open', 'Unresolved']",  
    "owned by": "(:Person)-[:OWNS]->(:Application)"  
  },  
  "similarExamples": [  
    {  
      "question": "Show external apps with critical open vulnerabilities",  
      "cypherPattern": "MATCH (a:Application)-[:HAS_VULNERABILITY]->(v:Vulnerability) ..."  
    }  
  ]  
}
```

5.5 Text-to-Cypher Agent

Purpose: generate read-only Cypher using only the retrieved schema, resolved entities, semantic mappings, and examples.

Example Cypher:

```
MATCH (p:Person)-[:OWNS]->(a:Application)-[:HAS_VULNERABILITY]->(v:Vulnerability)  
WHERE toLower(p.name) CONTAINS toLower($ownerName)  
AND a.exposure IN $exposureValues  
AND v.severity = $severity  
AND v.status IN $statuses  
RETURN
```

```
a.name AS application,  
a.appId AS appId,  
a.businessUnit AS businessUnit,  
collect(DISTINCT v.cveId)[0..10] AS sampleCVEs,  
count(DISTINCT v) AS vulnerabilityCount  
ORDER BY vulnerabilityCount DESC  
LIMIT 25
```

5.6 Cypher Validation Agent

Purpose: validate Cypher before execution.

This should be implemented mostly as deterministic code, not as a pure LLM prompt.

Validation checks:

- Read-only only
- No CREATE / MERGE / SET / DELETE
- No admin procedures
- No unrestricted APOC
- Allowed labels only
- Allowed relationships only
- Allowed properties only
- LIMIT required
- Timeout required
- Tenant filter required
- No unbounded traversal
- No obvious Cartesian product
- Parameterized values required

Example invalid output:

```
{  
  "valid": false,  
  "safeToExecute": false,  
  "errors": [  
    "Unknown label: App",  
    "Unknown relationship: HAS_CVE",  
    "Missing LIMIT"  
  ],  
  "repairable": true  
}
```

5.7 Access Policy Agent

Purpose: apply enterprise access controls before execution.

It injects:

tenant filter
business unit filter
region filter
role-based restrictions
sensitive-field masking
row-level access rules

Example policy injection:

```
AND a.tenantId = $tenantId  
AND a.businessUnit IN $allowedBusinessUnits
```

5.8 Result Verification Agent

Purpose: verify if Neo4j results actually answer the user question.

Checks:

Did the result answer the question?
Was result empty?
Was entity resolution ambiguous?
Were required filters applied?
Are duplicates suspicious?
Was relationship direction correct?
Is result too broad?

Example empty-result output:

```
{  
  "status": "empty_result",  
  "answerable": false,  
  "possibleCauses": [  
    "Owner name may not be resolved correctly",  
    "Exposure values may be wrong",  
    "Ownership may be through Team instead of Person",  
    "Relationship direction may be wrong"  
  ],  
  "nextAction": "diagnose"  
}
```

5.9 Answer Generation Agent

Purpose: produce a grounded answer using only verified results.

Rules:

Do not invent facts.
Mention uncertainty when confidence is low.
Use graph result as source of truth.
Include warnings if result may be incomplete.

5.10 Cypher Repair Agent

Purpose: fix invalid Cypher or wrong schema mapping.

Example repair mapping:

App -> Application
CVE -> Vulnerability
HAS_CVE -> HAS_VULNERABILITY

Rules:

Max 2 repair attempts
Never bypass validator
Never execute unvalidated query
Escalate if uncertain

5.11 Diagnosis Agent

Purpose: diagnose why results are empty or suspicious.

Example diagnostic checks:

```
MATCH (p:Person)
WHERE toLower(p.name) CONTAINS toLower($ownerName)
RETURN p.name, p.email
LIMIT 10
```

```
MATCH (a:Application)
RETURN DISTINCT a.exposure
LIMIT 20
```

5.12 Schema Issue Agent

Purpose: create governance issues when graph schema, aliases, or data quality are incomplete.

Example issue:

```
{
  "issueType": "MISSING_ALIAS",
  "userTerm": "CVE",
  "suggestedMapping": "CVE -> Vulnerability",
  "confidence": 0.94,
```

```
"requiresHumanReview": false  
}
```

6. End-to-End Flow

6.1 Normal Flow

1. User asks a natural-language question.
2. API Gateway authenticates user and forwards request to Orchestrator.
3. Intent Router Agent classifies question.
4. Entity Resolution Agent resolves entities and constraints.
5. Schema Retrieval Agent retrieves relevant graph schema.
6. Vector Retrieval Agent retrieves aliases, glossary, docs, and Cypher examples.
7. Context Package Builder assembles schema, mappings, examples, and safety rules.
8. Text-to-Cypher Agent generates read-only Cypher.
9. Cypher Validation Agent validates safety and schema correctness.
10. Access Policy Agent injects tenant and entitlement filters.
11. Read-only Neo4j Executor executes validated query.
12. Result Verification Agent checks result quality.
13. Answer Generation Agent produces grounded answer.
14. Orchestrator returns final response to user.

6.2 Failure Flow

Invalid Cypher

- > Cypher Validation Agent returns errors
- > Cypher Repair Agent repairs query
- > Cypher Validation Agent revalidates
- > Execution only happens if valid

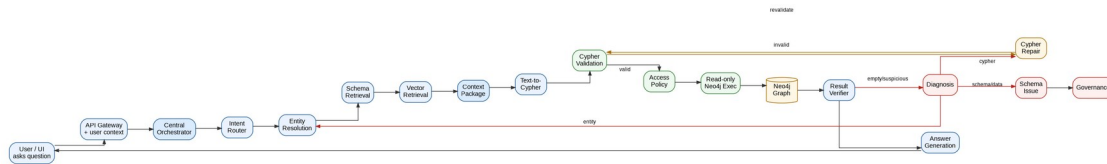
Empty or suspicious result

- > Result Verification Agent routes to Diagnosis Agent
- > Diagnosis Agent checks entity mapping, schema mapping, property values, and relationship direction
- > Orchestrator retries bounded paths or returns warning

Missing schema/data

- > Schema Issue Agent creates issue
- > Governance Queue receives issue for review
- > User receives best available answer or explanation

7. Sequence Diagram



End-to-End Multi-Agent Sequence

7.1 Mermaid Source

sequenceDiagram
autonumber

participant User as User / App UI
participant API as GraphRAG API Gateway
participant Orch as Central Orchestrator
participant Intent as Intent Router Agent
participant Entity as Entity Resolution Agent
participant Schema as Schema Retrieval Agent
participant Vector as Vector Retrieval Agent
participant VDB as Vector Database
participant Cypher as Text-to-Cypher Agent
participant Validator as Cypher Validation Agent
participant Repair as Cypher Repair Agent
participant Policy as Access Policy Agent
participant Executor as Read-only Neo4j Executor
participant Neo4j as Neo4j Knowledge Graph
participant Verifier as Result Verification Agent
participant Diagnosis as Diagnosis Agent
participant SchemaIssue as Schema Issue Agent
participant Governance as Governance Queue
participant Answer as Answer Generation Agent

User->>API: Ask natural language question
API->>Orch: Send question + user context
Orch->>Intent: Classify question intent
Intent-->>Orch: hybrid_graphrag / graph_query / vector_rag
Orch->>Entity: Resolve entities and business terms
Entity-->>Orch: Resolved entities and constraints
Orch->>Schema: Retrieve relevant graph schema
Schema-->>Orch: Labels, relationships, properties
Orch->>Vector: Retrieve semantic context
Vector->>VDB: Search aliases, glossary, docs, examples
VDB-->>Vector: Semantic matches

Vector-->>Orch: Mappings + examples + context
Orch-->>Cypher: Generate read-only Cypher
Cypher-->>Orch: Cypher + parameters
Orch-->>Validator: Validate Cypher safety and schema correctness

alt Cypher is invalid

Validator-->>Orch: Validation errors
Orch-->>Repair: Repair Cypher using schema + aliases
Repair-->>Orch: Repaired Cypher
Orch-->>Validator: Re-validate repaired Cypher
Validator-->>Orch: Validated Cypher

else Cypher is valid

Validator-->>Orch: Validated Cypher

end

Orch-->>Policy: Apply access policy
Policy-->>Orch: Tenant / BU / entitlement filters added
Orch-->>Executor: Execute validated read-only Cypher
Executor-->>Neo4j: Run Cypher query
Neo4j-->>Executor: Structured graph results
Executor-->>Orch: Query results
Orch-->>Verifier: Verify result quality

alt Result is good

Verifier-->>Orch: Good result / answerable
Orch-->>Answer: Generate grounded answer
Answer-->>Orch: Final answer
Orch-->>API: Return answer
API-->>User: Display final answer

else Result is empty or suspicious

Verifier-->>Orch: Empty / suspicious result
Orch-->>Diagnosis: Diagnose failure reason

alt Entity resolution issue

Diagnosis-->>Orch: Retry entity resolution
Orch-->>Entity: Re-resolve entity

else Cypher/schema mapping issue

Diagnosis-->>Orch: Repair Cypher
Orch-->>Repair: Repair query

else Missing schema or data issue

Diagnosis-->>Orch: Create issue
Orch-->>SchemaIssue: Create issue

SchemaIssue-->>Governance: Submit for human review

end

```
Orch-->>Answer: Generate best available response with warning
Answer-->>Orch: Final response
Orch-->>API: Return response
API-->>User: Display response
end
```

8. Skills and MCP Tools

8.1 Skills

```
graph.intent.classify
graph.entity.resolve
graph.schema.retrieve
graph.vector.retrieve
graph.cypher.generate
graph.cypher.validate
graph.policy.apply
graph.query.execute
graph.result.verify
graph.cypher.repair
graph.diagnose.failure
graph.schema.issue.create
graph.answer.generate
graph.feedback.capture
graph.eval.run
```

8.2 MCP Tools

MCP Tool	Purpose
neo4j.read.run_cypher	Execute validated read-only Cypher
neo4j.read.search_nodes	Search nodes by label/property
neo4j.read.expand_neighbors	Expand graph neighbors for a node
neo4j.schema.get_contract	Get curated schema contract
neo4j.schema.get_allowed_patterns	Get approved graph patterns
vector.search.semantic	Search vector DB indexes
alias.lookup_term	Map business term to graph term
alias.propose_mapping	Propose alias for review
cypher.validate.readonly	Validate read-only/schema-safe Cypher
policy.get_entitlements	Retrieve user entitlements
policy.inject_filters	Inject access filters into query

MCP Tool	Purpose
audit.log_event	Log all orchestration events
governance.create_issue	Create schema or data quality issue

9. Orchestrator State

The orchestrator should maintain this state for each request:

```
{
  "traceId": "trace-001",
  "question": "Which internet-facing applications owned by Ravi have critical CVEs?",
  "userContext": {
    "userId": "u123",
    "tenantId": "cyber",
    "allowedBusinessUnits": ["Digital Payments"]
  },
  "intent": null,
  "entities": {},
  "schemaContext": {},
  "vectorContext": [],
  "generatedCypher": null,
  "cypherParams": {},
  "validationResult": null,
  "policyAppliedCypher": null,
  "executionResult": [],
  "resultQuality": null,
  "repairAttempts": 0,
  "diagnosisAttempts": 0,
  "schemaIssues": [],
  "warnings": [],
  "finalAnswer": null
}
```

10. Runtime Rules and Guardrails

10.1 Hard Rules

No agent can directly execute Cypher.

Only the Neo4j Executor can execute Cypher.

Only validated Cypher can be executed.

All queries must be read-only.

All queries must include entitlement filters.

All queries must be logged.

Repair agent cannot bypass validation.

Schema issue agent cannot mutate production schema directly.

10.2 Retry Limits

max_cypher_repair_attempts: 2

max_entity_resolution_attempts: 2

max_diagnosis_attempts: 2

max_total_loop_depth: 5

query_timeout_seconds: 10

max_rows: 500

default_limit: 25

10.3 Blocked Cypher Clauses

CREATE

MERGE

SET

DELETE

DETACH DELETE

REMOVE

DROP

LOAD CSV

CALL dbms

CALL apoc.create

CALL apoc.merge

CALL apoc.periodic

11. Deployment View

User UI / API Client

-> GraphRAG API Gateway

-> Orchestrator Service

-> LLM Gateway

-> Vector DB

-> Neo4j

-> IAM / Policy Service

-> Audit Store

-> Governance Queue

Recommended stack:

API: FastAPI

Orchestration: LangGraph or custom deterministic state machine

Graph: Neo4j

Vector DB: existing vector database
LLM: enterprise LLM gateway
Policy: IAM / entitlement service
Audit: Postgres, Splunk, Elastic, or OpenTelemetry
Governance: Jira, ServiceNow, or internal workflow

12. MVP Scope

12.1 MVP Agents

1. Intent Router Agent
2. Entity Resolution Agent
3. Schema Retrieval Agent
4. Vector Retrieval Agent
5. Text-to-Cypher Agent
6. Cypher Validation Agent
7. Access Policy Agent
8. Graph Query Execution Service
9. Result Verification Agent
10. Answer Generation Agent
11. Cypher Repair Agent

12.2 Add Later

12. Diagnosis Agent
13. Schema Issue Agent
14. Feedback Learning Agent
15. Evaluation Agent

13. Acceptance Criteria

13.1 Functional

System answers at least 50 golden graph questions.
System generates valid Cypher for at least 85% of supported MVP questions.
System repairs at least 60% of simple label/relationship mistakes.
System blocks 100% of write/delete/admin Cypher attempts.
System applies tenant and BU filters to every graph query.
System returns grounded answers using verified Neo4j results only.

13.2 Quality

Cypher validity rate: >85% MVP, >95% production.
Answer correctness: >80% MVP, >90% production.
Hallucinated field/relationship rate: <5%.

Empty result diagnosis present when row count = 0.
P95 latency: <15 seconds MVP.

13.3 Security

No direct LLM-to-Neo4j execution.
No write queries allowed.
No unrestricted APOC/admin calls.
All queries logged with trace ID.
All graph queries use a read-only account.

14. Final Design Summary

The final multi-agent design is:

Central Orchestrator

- > Intent Router Agent
- > Entity Resolution Agent
- > Schema Retrieval Agent + Vector Retrieval Agent
- > Context Package Builder
- > Text-to-Cypher Agent
- > Cypher Validation Agent
- > Access Policy Agent
- > Read-only Neo4j Executor
- > Result Verification Agent
- > Answer Generation Agent

Failure loop:

Invalid Cypher -> Cypher Repair Agent -> Validate again

Empty result -> Diagnosis Agent -> Entity retry / Cypher repair / schema issue

Missing schema/data -> Schema Issue Agent -> Governance queue

The most important principle:

Agents can reason and propose.
Only controlled services can execute.
Governance approves schema/data changes.